

Title: Senior ML Scientist Take-Home coding task: Robust Toxicity Classification

Scenario: We are processing high-volume user comments. We need a custom, lightweight Transformer model to flag toxic comments in our data. We cannot use off-the-shelf pre-trained weights because we need full control over the architecture for future optimisations.

The Data: Use the [Civil Comments Dataset](#). Treat target ≥ 0.5 as "Toxic".

Requirements:

1. **Custom Architecture:** Implement a small Transformer using pre-layer normalisation from scratch using standard PyTorch layers (like `nn.Linear`, `nn.MultiheadAttention`, `nn.LayerNorm`, etc.)
2. **Engineering & MLOps:**
 - Implement a training loop that can be run on one or multiple GPUs if available on other machines.
 - Optional: Include a Dockerfile and structure the code as a proper Python package (`setup.py/pyproject.toml`).
3. **Data Pipeline:**
 - Efficient data loading (demonstrate your choice of handling tokenisation on the fly vs. pre-processing).
 - Strict train/val/test split or stratified sampling.

Deliverables:

A light-weight zip file containing:

- A well organised repo including the code (including a readme file with instructions on how to use the repo, but no data or models should be included).
- A short `report.md` explaining:
 - *Explain choice of architecture, Layer Normalisation and Loss function and hyper-parameters (vocab size, embedding dim, number of layers, etc.).*
 - How you handled the class imbalance.
 - Optional: A "Post-Mortem": If you had to deploy this to handle 10k requests/second, what would you change?

Note: You are encouraged to use AI coding assistants. You have 7 days from receiving this task to complete it and submit your zip folder. During the follow up interview, we will look at your code on your screen, and go through it together and look at the git history and potentially ask to make small changes. We evaluate it based on the **decisions** you made and your ability to explain *why* the generated code is correct. The model does not need to be trained nor use the whole dataset, and we will not judge based on the model performance, but it needs to run and work without errors. We are keen to see your choices, and the logic behind them and discuss the model performance metrics.